

Ensemble Learning and Boosting

COMP3314
Introduction to Machine Learning

Dong Xu

Outline

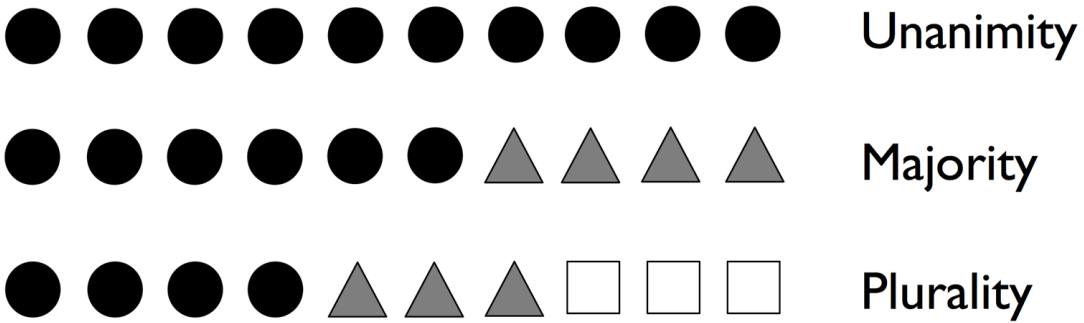
- A set of classifiers can often have a better predictive performance than any of its individual members
- We will learn how to do the following
  - Make predictions based on majority voting
  - Use bagging to reduce overfitting by drawing random combinations of the training set with repetition
  - Apply boosting to build powerful models from weak learners that learn from their mistakes

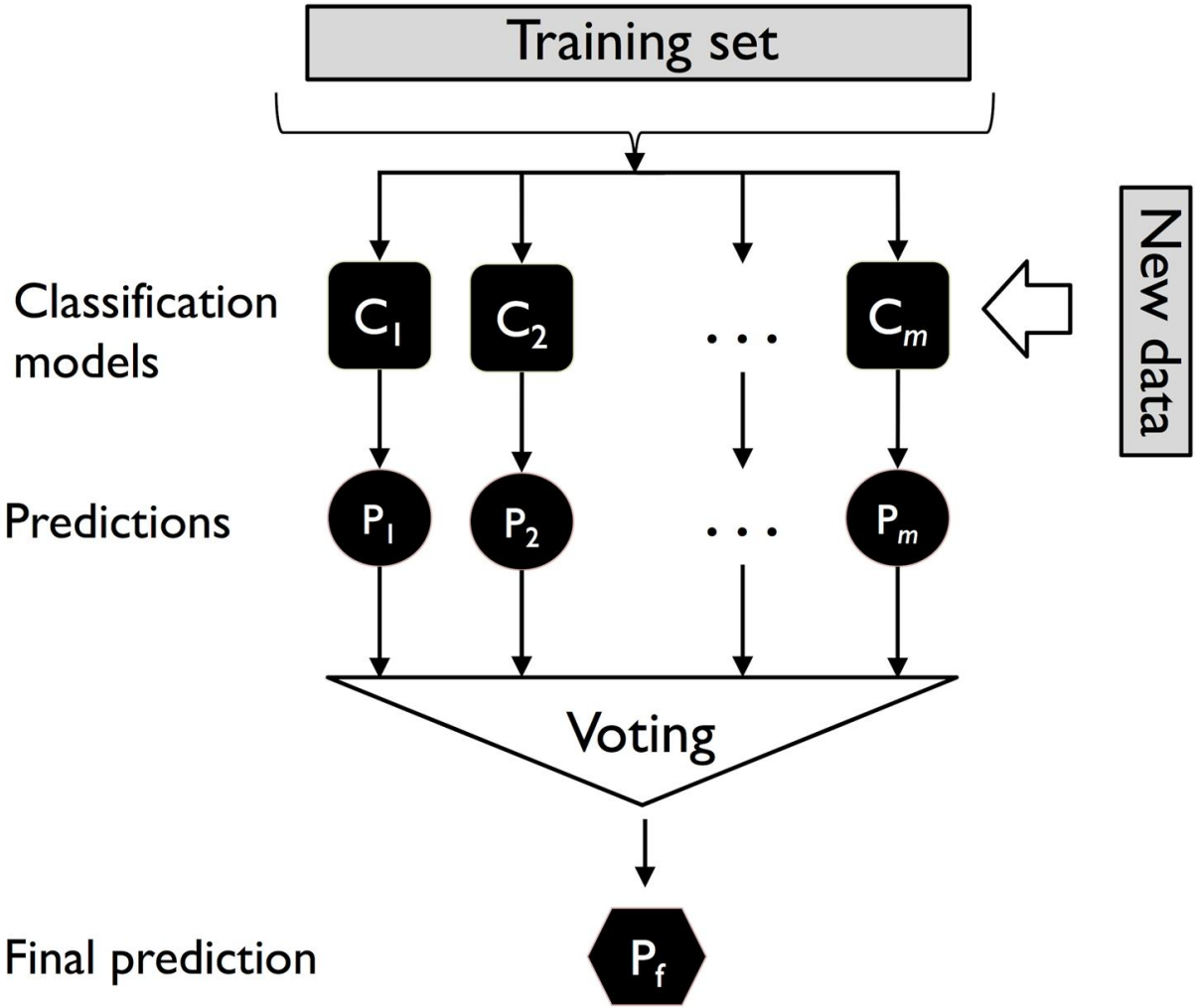
Learning with Ensembles

- Goal
  - Combine different classifiers into a meta-classifier that has better generalization performance than each individual classifier alone
- E.g., assuming that we collected predictions from 10 experts
  - Ensemble methods would allow us to strategically combine these predictions by the 10 experts to come up with a prediction that is more accurate and robust than the predictions by each individual expert

Majority/Plurality Voting

- In this chapter we will focus on the most popular ensemble methods that use the majority voting principle
  - Majority voting simply means that we select the class label that has been predicted by the majority of classifiers, that is, received more than 50 percent of the votes
- Majority vote refers to binary class settings only
  - However, it is easy to generalize the majority voting principle to multi-class settings, which is called plurality voting
    - Select the class label that received the most votes (mode)




Training set
Classification models
C_1
C_2
C_m
New data
Predictions
P_1
P_2
P_m
Voting
Final prediction
P_f

Majority/Plurality Voting

- To predict a class label via simple majority or plurality voting, we combine the predicted class labels of each individual classifier, C_j and select the class label, $\hat{y}$ that received the most votes

$$\hat{y} = mode\left\{C_1(\boldsymbol{x}), C_2(\boldsymbol{x}), \ldots, C_m(\boldsymbol{x})\right\}$$

- E.g., in a binary classification task where class_1 = −1 and class_2 = +1, we can write the majority vote prediction as follows

$$C(\boldsymbol{x}) = sign\left[\sum_j^m C_j(\boldsymbol{x})\right] = \begin{cases} 1 \ \ if \sum_i C_j(\boldsymbol{x}) \geq 0 \\ -1 \ otherwise \end{cases}$$

Weighted Majority Vote Classifier

- Consider the following simple weighted majority vote classifier

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j \chi_A \left(C_j \left(\boldsymbol{x} \right) = i \right)$$

- Here,
  - w_j is a weight associated with a base classifier C_j ,
  - $\hat{y}$ is the predicted class label of the ensemble,
  - χ_A (Greek chi) is a function that gives you the prediction $[C_j(x) = i \in A]$
  - A is the set of unique class labels
- For equal weights, we can simplify this equation and write it as follows

$$\hat{y} = mode\left\{ C_1 \left(\boldsymbol{x} \right), C_2 \left(\boldsymbol{x} \right), \ldots, C_m \left(\boldsymbol{x} \right) \right\}$$

Example

- Consider an ensemble of three base classifiers, C_j ($j \in \{1, 2, 3\}$)
  - Two classifiers predict the class label 0, and one classifier C_3 predicts that the sample belongs to class 1
- If we weight the predictions of each base classifier equally, the majority vote would predict that the sample belongs to class 0

$$C_1(\boldsymbol{x}) \rightarrow 0,\ C_2(\boldsymbol{x}) \rightarrow 0,\ C_3(\boldsymbol{x}) \rightarrow 1$$

$$\hat{y} = mode\{0,0,1\} = 0$$

- Now, let us assign a weight of 0.6 to C_3 and a weight of 0.2 to C_1 and C_2

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j \chi_A\left(C_j(\boldsymbol{x}) = i\right)$$

$$= \arg\max_i \left[0.2 \times i_0 + 0.2 \times i_0 + 0.6 \times i_1\right] = 1$$

```
import numpy as np
np.argmax(np.bincount([0, 0, 1],
          weights=[0.2, 0.2, 0.6]))
```

```
1
```

Probabilities

- Recall that certain classifiers can return the probability of a predicted class label
  - In scikit-learn: via the predict_proba method
- Using the predicted class probabilities instead of the class labels for majority voting can be useful if the classifiers in our ensemble are well calibrated
- The modified version of the majority vote for predicting class labels from probabilities can be written as follows

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j p_{ij}$$

- Here, p_{ij} is the predicted probability of the jth classifier for class label i

Example

$$\hat{y} = \arg\max_i \sum_{j=1}^{m} w_j p_{ij}$$

- We have a binary classification problem with class labels $i \in \{0, 1\}$ and an ensemble of three classifiers C_j ($j \in \{1,2,3\}$)
- The classifiers C_j return the following class membership probabilities for a particular sample x

$$C_1(\boldsymbol{x}) \rightarrow [0.9, 0.1],\ C_2(\boldsymbol{x}) \rightarrow [0.8, 0.2],\ C_3(\boldsymbol{x}) \rightarrow [0.4, 0.6]$$

- We can then calculate the individual class probabilities as follows

$$p(i_0 \mid \boldsymbol{x}) = 0.2 \times 0.9 + 0.2 \times 0.8 + 0.6 \times 0.4 = 0.58$$

$$p(i_1 \mid \boldsymbol{x}) = 0.2 \times 0.1 + 0.2 \times 0.2 + 0.6 \times 0.6 = 0.42$$

$$\hat{y} = \arg\max_i \left[p(i_0 \mid \boldsymbol{x}), p(i_1 \mid \boldsymbol{x}) \right] = 0$$

```
ex = np.array([[0.9, 0.1],
               [0.8, 0.2],
               [0.4, 0.6]])
p = np.average(ex, axis=0,
               weights=[0.2, 0.2, 0.6])
p
```

```
array([0.58, 0.42])
```

Methods for Constructing Ensembles

Subsampling the training examples

Multiple hypotheses are generated by training individual classifiers on different datasets obtained by resampling a common training set (Bagging, Boosting).

Manipulating the input features

Multiple hypotheses are generated by training individual classifiers on different representations, or different subsets of a common feature vector.

Manipulating the output targets

The output targets for C classes are encoded with an L-bit codeword, and an individual classifier is built to predict each one of the bits in the codeword.

Additional "auxiliary" targets may be used to differentiate classifiers.

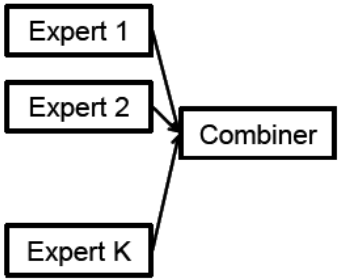
Modifying the learning parameters of the classifier

A number of classifiers are built with different learning parameters, such as number of neighbors in a k Nearest Neighbor rule, initial weights in an MLP, etc.

Structure of Ensemble Classifiers

Parallel

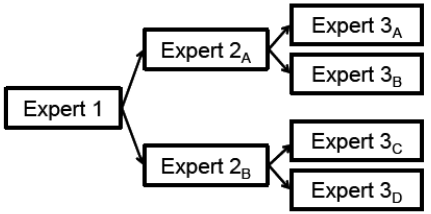
All the individual classifiers are invoked independently, and their results are fused with a combination rule (e.g., average, weighted voting) or a meta-classifier (e.g.,stacked generalization)
The majority of ensemble approaches in the literature fall under this category



Cascading or Hierarchical

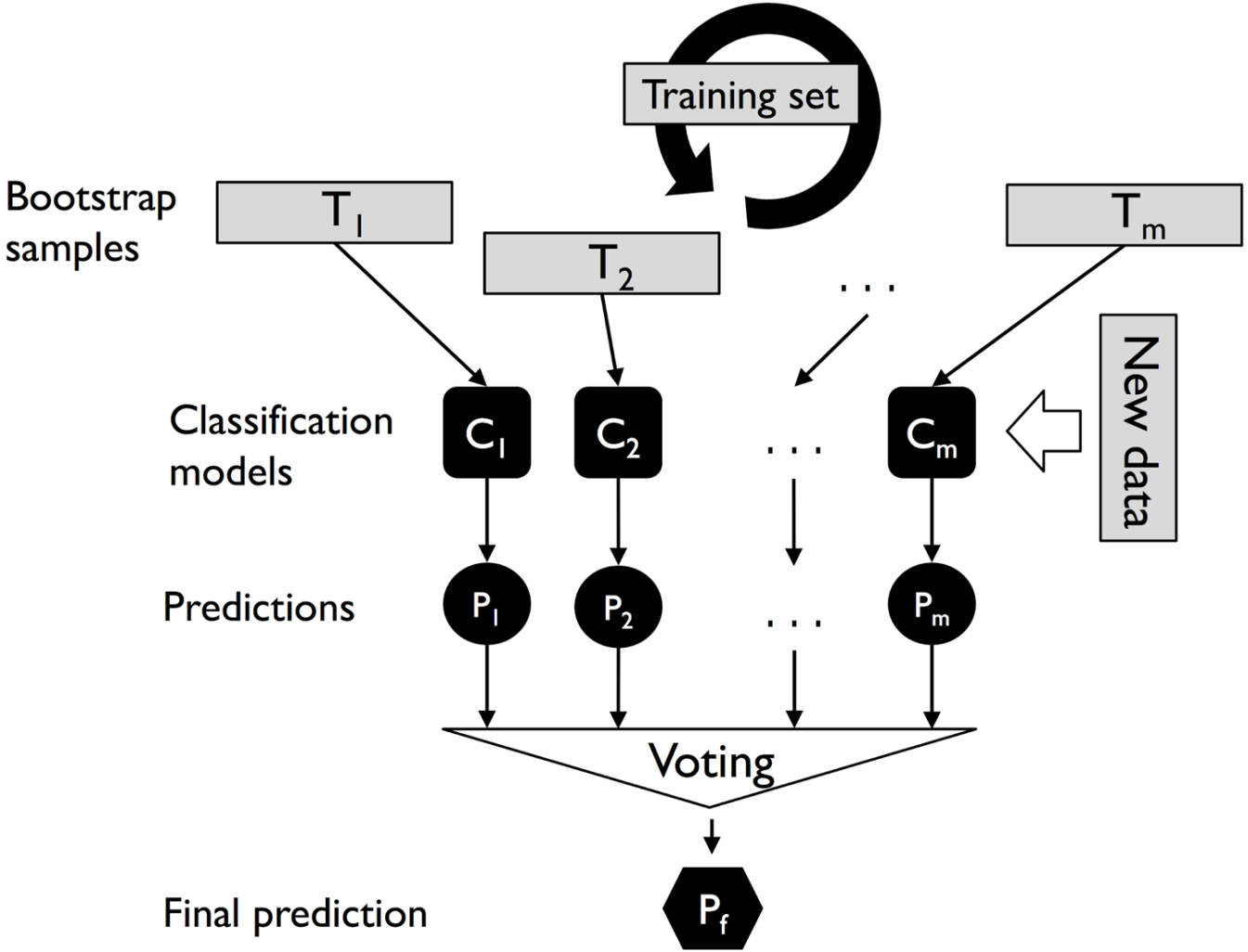
Classifiers are invoked in a sequential or tree-structured fashion
For the purpose of efficiency, inaccurate but fast methods are invoked first (maybe using a small subset of the features), and computationally more intensive but accurate methods are left for the latter stages





Outline

- In this chapter, we will construct a set of classifiers that can often have a better predictive performance than any of its individual members
- We will learn how to do the following
  - Make predictions based on majority voting
  - Use bagging to reduce overfitting by drawing random combinations of the training set with repetition
  - Apply boosting to build powerful models from weak learners that learn from their mistakes


Training set
Bootstrap samples
T_1
T_2
. . .
T_m
Classification models
C_1
C_2
. . .
C_m
New data
Predictions
P_1
P_2
. . .
P_m
Voting
Final prediction
P_f

Bagging

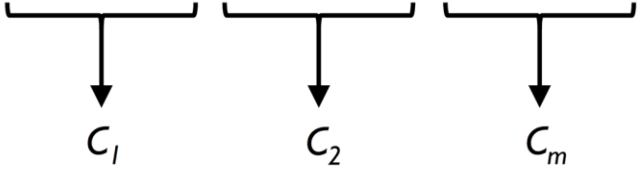
- Bagging is an ensemble learning technique that is closely related to the MajorityVoteClassifier implemented previously
- Instead of using the same training set to fit the individual classifiers in the ensemble, we draw bootstrap samples (random samples with replacement) from the initial training set
- Bagging is also known as bootstrap aggregating
- Bagging was first proposed by Leo Breiman in 1994
  - He showed that bagging can improve the accuracy of unstable models and decrease the degree of overfitting
  - Recommend reading:
    Bagging predictors, L. Breiman, Technical Report
    - > 30 000 citations



Bagging in a Nutshell

- Seven different training instances
  - 1 to 7
- Sampled randomly with replacement in each round of bagging
- Each bootstrap sample is then used to fit a classifier C_j
- Note that each subset contains a certain portion of duplicates and some of the original samples don't appear in a resampled dataset at all
- Once the individual classifiers are fit to the bootstrap samples, the predictions are combined using majority voting

Sample indices	Bagging round 1	Bagging round 2	…
1	2	7	…
2	2	3	…
3	1	2	…
4	3	1	…
5	7	1	…
6	2	7	…
7	4	7	…



Bagging - Conclusion

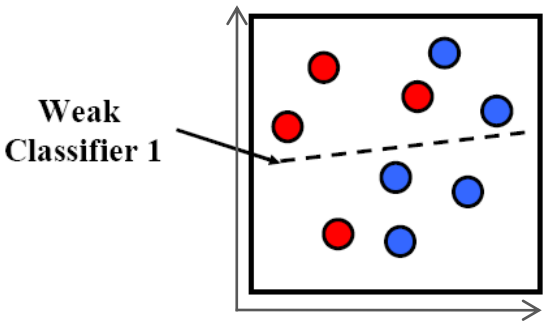
- In practice, more complex classification tasks and a dataset's high dimensionality can often lead to overfitting in single decision tree
- This is where the bagging algorithm can really play to its strengths
  - It can be an effective approach to reduce the variance of a model
- However, bagging is ineffective in reducing model bias, that is, models that are too simple to capture the trend in the data well
- This is why we want to perform bagging on an ensemble of classifiers with low bias
  - E.g., unpruned decision trees

Outline

- We will construct a set of classifiers that can often have a better predictive performance than any of its individual members
- We will learn how to do the following
  - Make predictions based on majority voting
  - Use bagging to reduce overfitting by drawing random combinations of the training set with repetition
  - Apply boosting to build powerful models from weak learners that learn from their mistakes

Boosting [Schapire, 1990; Freund and Schapire, 1996]

- Boosting takes a different resampling approach than bagging, which maintains a constant probability of 1/N for selecting each individual example
- In boosting, this probability is adapted over time based on performance
- The component classifiers are built sequentially, and examples that are mislabeled by previous components are chosen more often than those that are correctly classified
- Boosting is based on the concept of a "weak learner", an algorithm that performs slightly better than chance (e.g., 50% classification rate on binary tasks)
- Schapire has shown that a weak learner can be converted into a strong learner by changing the distribution of training examples
- Boosting can also be used with classifiers that are highly accurate, but the benefits in this case will be very small

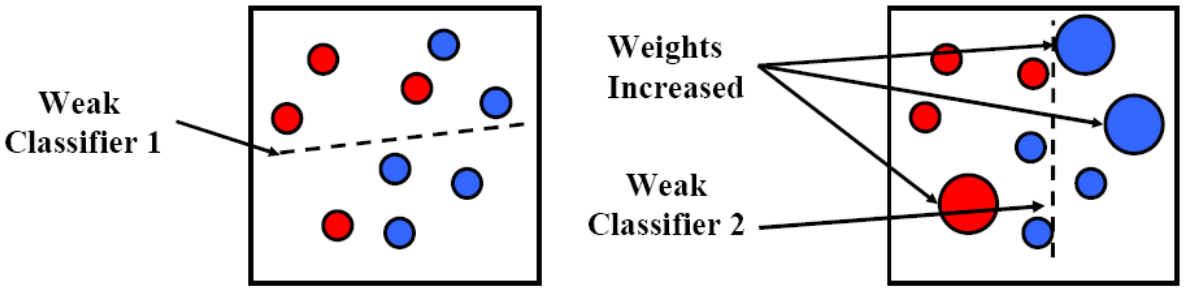


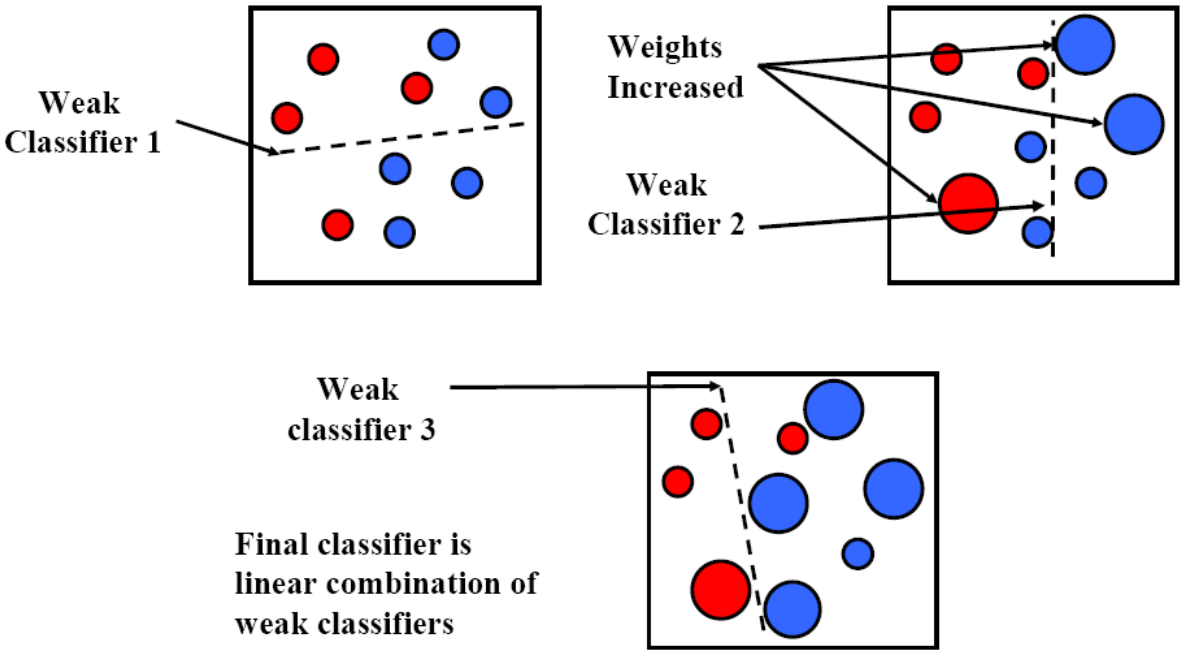
Consider a 2-d feature space with positive and negative examples.

Each weak classifier splits the training examples with at least 50% accuracy.

Examples misclassified by a previous weak learner are given more emphasis at future rounds.

Figure adapted from Freund and Schapire


Weak
Classifier 1
Weights
Increased
Weak
Classifier 2


Weak
Classifier 1
Weights
Increased
Weak
Classifier 2
Weak
classifier 3
Final classifier is
linear combination of
weak classifiers

- Given example images $(x_1, y_1), \ldots, (x_n, y_n)$ where $y_i = 0, 1$ for negative and positive examples respectively.
- Initialize weights $w_{1,i} = \frac{1}{2m}, \frac{1}{2l}$ for $y_i = 0, 1$ respectively, where m and l are the number of negatives and positives respectively.
- For $t = 1, \ldots, T$:

  1. Normalize the weights,

     $$w_{t,i} \leftarrow \frac{w_{t,i}}{\sum_{j=1}^{n} w_{t,j}}$$

     so that w_t is a probability distribution.

  2. For each feature, j, train a classifier h_j which is restricted to using a single feature. The error is evaluated with respect to w_t, $\epsilon_j = \sum_i w_i \left| h_j(x_i) - y_i \right|$.
  3. Choose the classifier, h_t, with the lowest error ϵ_t.
  4. Update the weights:

     $$w_{t+1,i} = w_{t,i} \beta_t^{1-e_i}$$

     where $e_i = 0$ if example x_i is classified correctly, $e_i = 1$ otherwise, and $\beta_t = \frac{\epsilon_t}{1-\epsilon_t}$.

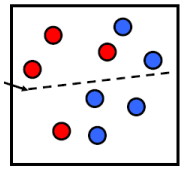
- The final strong classifier is:

$$h(x) = \begin{cases} 1 & \sum_{t=1}^{T} \alpha_t h_t(x) \geq \frac{1}{2} \sum_{t=1}^{T} \alpha_t \\ 0 & \text{otherwise} \end{cases}$$

where $\alpha_t = \log \frac{1}{\beta_t}$

Freund & Schapire 1995

Start with uniform weights on training examples



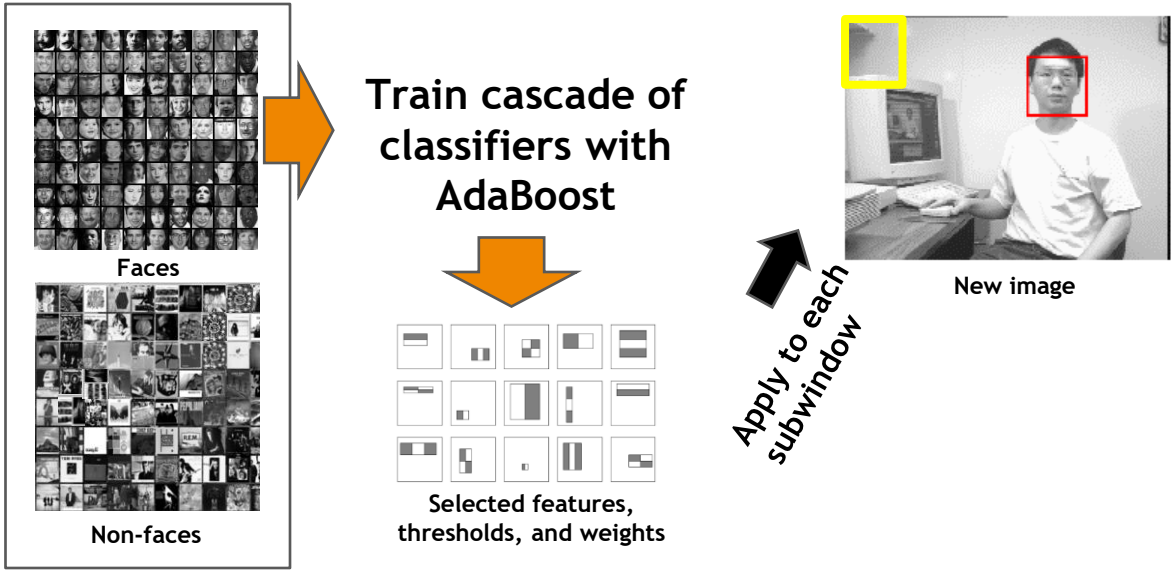
$\{x_1, \ldots x_n\}$

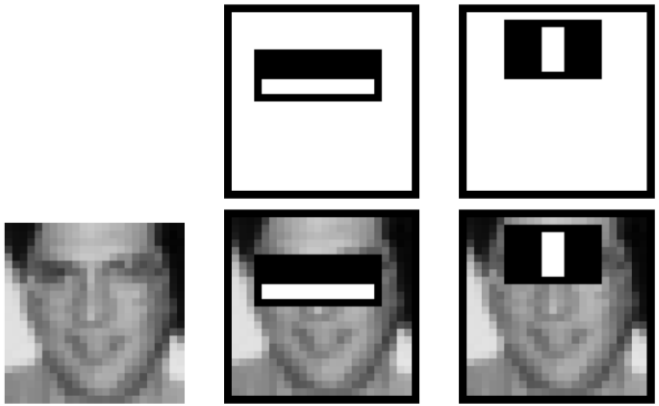
Evaluate *weighted* error for each feature, pick best.

Incorrectly classified -> more weight

Correctly classified -> less weight

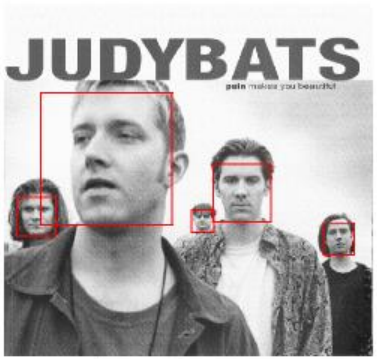
Final classifier is combination of the weak ones, weighted according to error they had.

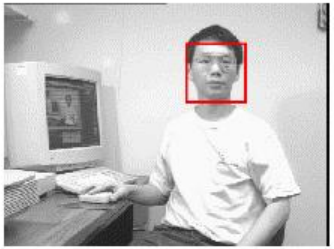

Faces
Non-faces
Train cascade of classifiers with AdaBoost
Selected features, thresholds, and weights
Apply to each subwindow
New image

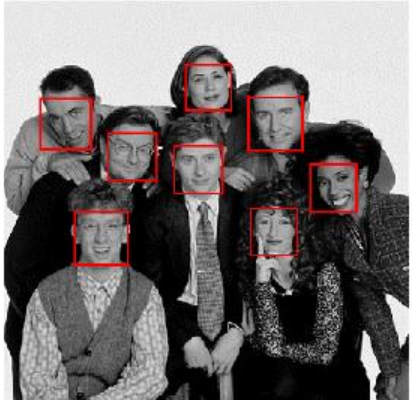


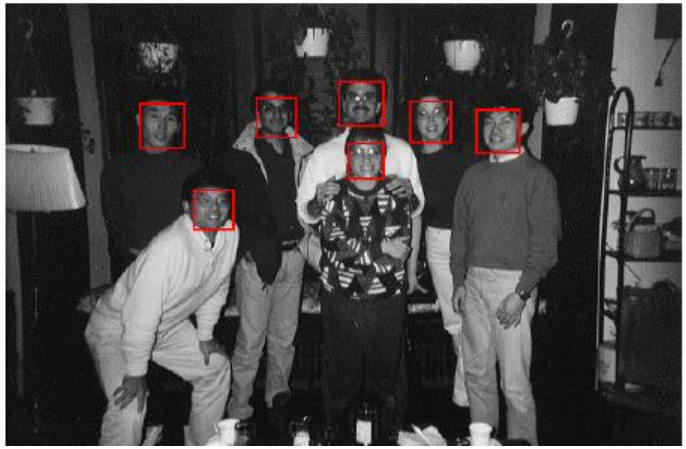
First two features selected

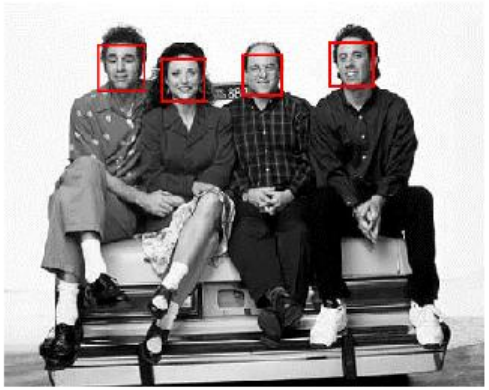


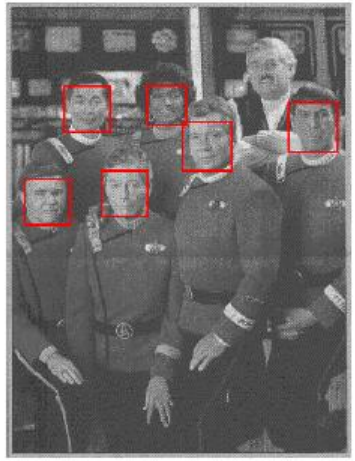

JUDYBATS
pain makes you beautiful

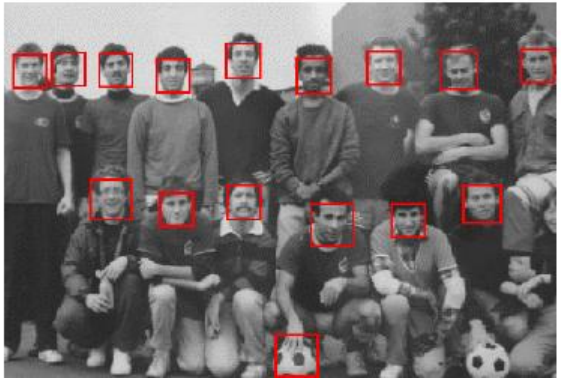


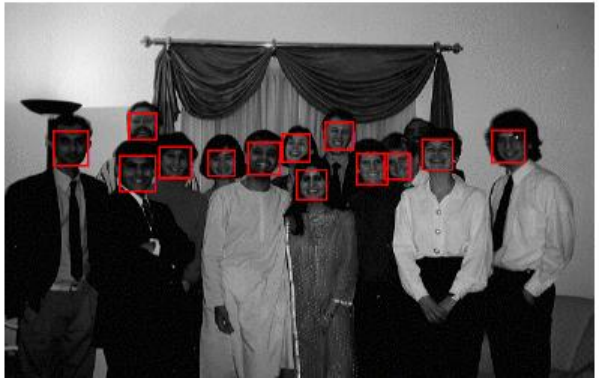






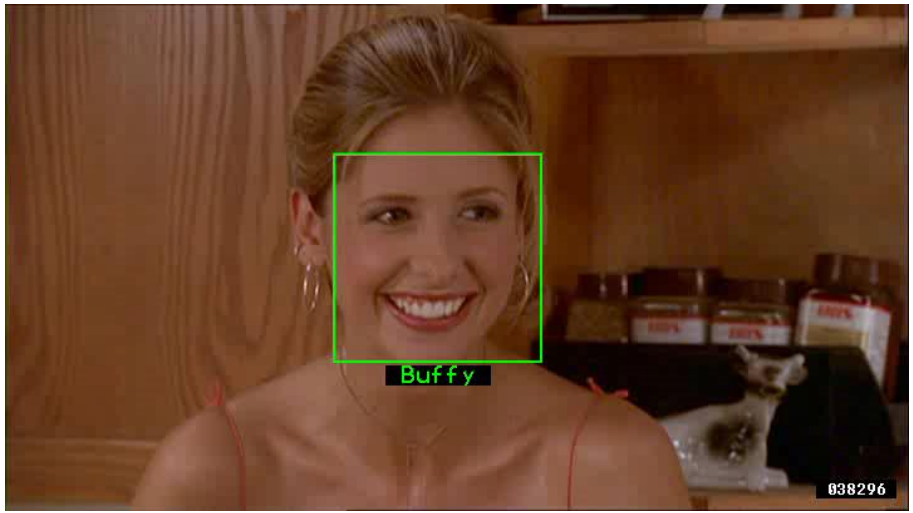






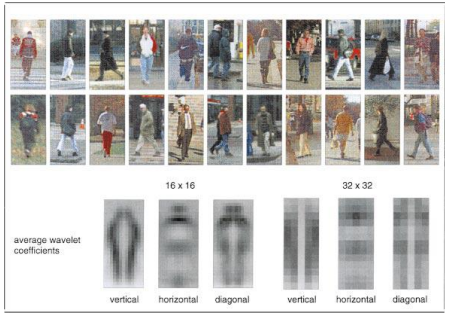






Frontal faces detected and then tracked, character names inferred with alignment of script and subtitles.

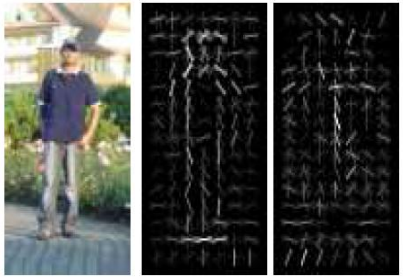
Everingham, M., Sivic, J. and Zisserman, A.
"Hello! My name is... Buffy" - Automatic naming of characters in TV video,
BMVC 2006.
http://www.robots.ox.ac.uk/~vgg/research/nface/index.html



SVM with Haar wavelets [Papageorgiou & Poggio, IJCV 2000]



Space-time rectangle features [Viola, Jones & Snow, ICCV 2003]



SVM with HoGs [Dalal & Triggs, CVPR 2005]